

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 3 Practical No: 1

Roll no: 35 PRN No: 12414739 Name of Student: KHYATI KENDHE

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset.

Algorithm :

1. Import the required libraries: TensorFlow and Matplotlib.
2. Load the Fashion-MNIST dataset from TensorFlow.
3. Normalize the image pixel values by dividing them by 255.0.
4. Display the shape of the training and testing datasets.
5. Visualize a sample image from the dataset.
6. Create a Sequential neural network model using Keras.
7. Add the Flatten and Dense layers to the model.
8. Compile the model using the Adam optimizer and sparse categorical cross-entropy loss function.
9. Train the model for 10 epochs.
10. Evaluate the model using the test dataset and display the accuracy.
11. Save the trained model in .keras format.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Load Fashion-MNIST dataset

(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()

# Data preprocessing and normalization

train_images = train_images / 255.0
test_images = test_images / 255.0

# Display dataset information

print(train_images.shape)
print(test_images.shape)
print(train_labels)

# Data visualization

plt.imshow(train_images[1], cmap='gray')
plt.show()

# Create the model

my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

# Compile the model

my_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model

my_model.fit(train_images, train_labels, epochs=10)

# Evaluate the model

val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print("Test Accuracy:", val_acc)
```

```
# Save the model
my_model.save("my_fashion_mnist_model.keras")
```

Output :

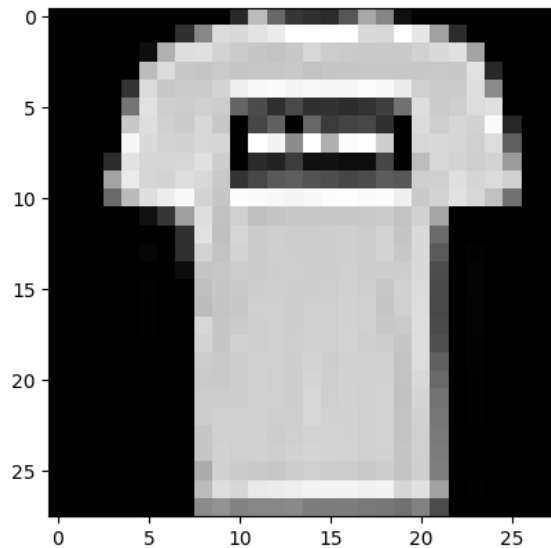
▶ # Visualizing data

```
print(train_images.shape)
print(test_images.shape)

print(train_labels)

plt.imshow(train_images[1], cmap='gray')
plt.show()
```

```
... (60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]
```



```
# Defining neural network model
```

```
my_model = tf.keras.models.Sequential()  
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))  
my_model.add(tf.keras.layers.Dense(128, activation='relu'))  
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

```
my_model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
# Training the model
```

```
my_model.fit(train_images, train_labels, epochs=10)
```

```
Epoch 1/10  
1875/1875 ————— 10s 5ms/step - accuracy: 0.6649 - loss: 1.0903  
Epoch 2/10  
1875/1875 ————— 9s 5ms/step - accuracy: 0.7693 - loss: 0.6420  
Epoch 3/10  
1875/1875 ————— 8s 4ms/step - accuracy: 0.7967 - loss: 0.5673  
Epoch 4/10  
1875/1875 ————— 9s 5ms/step - accuracy: 0.8139 - loss: 0.5249  
Epoch 5/10  
1875/1875 ————— 9s 5ms/step - accuracy: 0.8249 - loss: 0.4967  
Epoch 6/10  
1875/1875 ————— 8s 4ms/step - accuracy: 0.8319 - loss: 0.4770  
Epoch 7/10  
1875/1875 ————— 9s 5ms/step - accuracy: 0.8379 - loss: 0.4614  
Epoch 8/10  
1875/1875 ————— 11s 5ms/step - accuracy: 0.8428 - loss: 0.4494  
Epoch 9/10  
1875/1875 ————— 8s 4ms/step - accuracy: 0.8455 - loss: 0.4391  
Epoch 10/10  
1875/1875 ————— 9s 5ms/step - accuracy: 0.8482 - loss: 0.4307  
<keras.src.callbacks.history.History at 0x7b6dc40dcbc0>
```

```
# Checking model accuracy on test data
```

```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)  
print("Test accuracy:", val_acc)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.8345 - loss: 0.4598  
Test accuracy: 0.8345000147819519
```

```
# Saving the model
```

```
my_model.save("my_fashion_mnist_model.keras")
```